

Lecture 10

Geometry-Aware ML I

— Equivariance & Symmetry

AI for Geometry — MM845

Edward Hirst

Instituto de Matemática, Estatística e Computação Científica
Universidade Estadual de Campinas

`ehirst@unicamp.br`

Setembro 2026

Overview

1. Symmetry as Inductive Bias
2. Graph Neural Networks
3. Symmetry in Practice
4. Summary

Recap (Lecture 9): MDPs, Q-learning, policy gradients: learning as sequential decision-making.
Guiding question: *Given the symmetry group of your data, what is the right hypothesis space?*

The Thread We Have Been Pulling All Course

Definitions

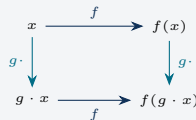
Let G act on input space X and output space Y .

A map $f : X \rightarrow Y$ is

invariant: $f(g \cdot x) = f(x)$,

equivariant: $f(g \cdot x) = g \cdot f(x)$,

for all $g \in G$.



Square commutes $\Rightarrow$ equivariant.

Replace the right edge by $\text{id} \Rightarrow$ invariant.

- Lecture 5's CNNs $\Rightarrow$ equivariance for $G = \mathbb{Z}^2$ (translations).
Lecture 6's Transformers $\Rightarrow$ equivariance for $G = S_N$ (permutations).
...both were instances of one design principle, stated today in general.
- **Benefit 1 — Data efficiency:** An equivariant model learns one G -orbit at once; a generic model must see each orbit element separately. Effective sample size multiplies by $\sim |G|$ (or the orbit volume for Lie groups).
- **Benefit 2 — Correctness:** Predictions of geometric/physical quantities *must* respect the symmetry (a rotated molecule has rotated forces; a relabelled graph has the same genus). Equivariant architectures satisfy this *exactly*, not approximately.
- Generalisation view (Lecture 2): Equivariance shrinks the hypothesis space in precisely the right direction — bias without wrongness.

The General Design Principle

Equivariant deep learning

- Fix the group G of your data.
- Set G -representations on each layer space.
Build networks as

$$f = \sigma \circ L_k \circ \cdots \circ \sigma \circ L_1,$$

...with each L_i a G -equivariant linear map (an intertwiner),

...and σ *equivariance-compatible*: $\sigma(\rho(g)v) = \rho(g)\sigma(v)$

- For an invariant output, finish with a G -invariant pooling.

- The space of intertwiners $\text{Hom}_G(V_i, V_{i+1})$ is computable by Schur's lemma: decompose into irreducibles, match isotypic components: *representation theory as architecture design*.
- $G = \mathbb{Z}^2$: intertwiners = convolutions (Lecture 5's theorem — now a corollary).
- $G = S_N$ on datapoints $H \in \mathbb{R}^{N \times n}$ ($N = \#$ points in *this* input, $n = \text{dim}$ of each point):
 $\mathbb{R}^N \cong \mathbb{R}\mathbf{1} \oplus \{x : \sum_i x_i = 0\} \Rightarrow \text{Hom}_{S_N}(\mathbb{R}^N, \mathbb{R}^N) = \text{span}\{I, \mathbf{1}\mathbf{1}^\top\} \Rightarrow \text{self} + \text{global}$.
Layer: $H \mapsto \sigma(HW_1 + \mathbf{1}\mathbf{1}^\top HW_2)$, only $W_1, W_2 \in \mathbb{R}^{n \times n}$ learned: **DeepSets**.
...make the $N \times N$ part data-dependent instead of fixed: **attention**.
- $G = S_N$ on datapoints H and neighbourhoods A (simultaneously equivariant):
 $\Rightarrow$ operator $\mathbf{1}\mathbf{1}^\top \rightarrow A \Rightarrow \text{self} + \text{neighbours}$.
Layer: $H \mapsto \sigma(HW_1 + AHW_2)$: **message passing**, a new operator not a new irrep.
... A is data, not part of a fixed representation (same group, larger input space).

Beyond Translations: Rotations and Lie Groups

- **Group CNNs** (finite case): to gain equivariance under the dihedral group (90° rotations + reflections of images), *lift* feature maps to functions on $\mathbb{Z}^2 \rtimes D_4$ and convolve over the group — exact equivariance for grid-compatible symmetries (extends to any finite G).
- **Steerable CNNs** (continuous case): for $SO(2)/SO(3)$, expand filters in irreducible-representation bases (circular/spherical harmonics); feature “fields” transform in prescribed representations — connections to induced representations and, geometrically, to sections of associated bundles.
- **Spherical CNNs**: data on S^2 (e.g. cosmology, molecular surfaces, directions): convolutions over $SO(3)$ computed via generalised FFTs.
- Cost–benefit: exact equivariance costs implementation complexity; for small groups, *data augmentation* (Lecture 4) approximates it cheaply (this trade-off is weighed properly later).

A feature *field*, not a feature vector

Applied representation theory:
Schur’s lemma, Peter–Weyl, induced representations
→ Machinery you know, deployed as engineering.
→ The choice of irrep *is* the declaration of how a feature transforms.



Positions move by g ; the arrows *also* turn, by $\rho(g)$. A scalar field’s numbers would not.

Graphs: Ubiquitous, and Permutation-Symmetric

Graphs as geometric data

Triangulations (1-skeleta or dual graphs), polytope face lattices, quivers, discretised manifolds, molecular structures: combinatorial geometry *is* graph data.

A graph = node features $H \in \mathbb{R}^{N \times n} + \text{adjacency } A \in \{0,1\}^{N \times N}$.

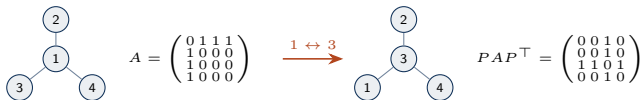
The naive implementation

→ Feed A (flattened) to an MLP, or treat A as an image for a CNN.

...vertex labelling is *arbitrary*, and S_N acts by $A \mapsto PAP^\top$.

→ The naive models see each relabelling as different data, with $N!$ relabellings, ...they will overfit orderings (Lecture 5 warning generalised).

- Requirement: node-level outputs equivariant ($H \mapsto PH$ when $A \mapsto PAP^\top$); graph-level outputs invariant.
- By the intertwiner analysis: essentially the only local linear equivariant operations combine *each node* with an *aggregate of its neighbours* $\Rightarrow$ message passing.



The *same* graph. An MLP on the flattened matrix sees two unrelated inputs; a GNN sees one.

Message Passing Neural Networks

The message passing layer

Each node v updates by aggregating over neighbours $\mathcal{N}(v)$:

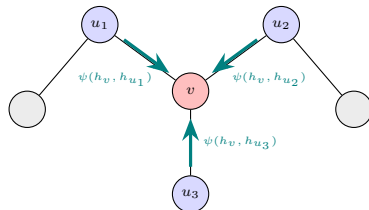
$$h_v^{(t+1)} = \phi \left(h_v^{(t)}, \bigoplus_{u \in \mathcal{N}(v)} \psi(h_v^{(t)}, h_u^{(t)}) \right)$$

with ψ = message function (small MLP);

$\bigoplus$ = *perm-invariant* aggregation (sum/mean/max);

ϕ = update function (small MLP).

- Equivariance is by construction: nothing refers to vertex labels.
- t rounds $\Rightarrow$ information flows t graph-steps: receptive fields again, now in graph metric. (Rounds = Layers)
- Graph-level readout: $\bigoplus_v h_v^{(T)}$ — invariant, any graph size.



Node v receives messages from its neighbours; a symmetric $\bigoplus$ forgets their order.

The GNN Zoo, and What GNNs Cannot See

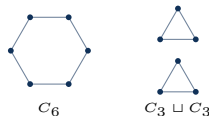
Common instantiations (specific choices of $\{\psi, \oplus, \phi\}$):

- **GCN:** $\psi = \hat{A}_{vu} h_u W$, $\oplus = \text{weighted sum}$, $\phi = \sigma$.
Interpretation: one step of Laplacian smoothing, a shared low-pass filter (cheap spectral motivation).
- **GAT:** $\psi = \alpha_{vu}(h_v, h_u) h_u W$, $\oplus = \text{weighted sum}$, $\phi = \sigma$.
Interpretation: edge importance becomes learned and data-dependent, not fixed by degree.
- **GIN:** $\psi = h_u$, $\oplus = \text{sum}$, $\phi = \text{MLP}((1 + \epsilon)h_v + \cdot)$.
Interpretation: matches WL colour-refinement exactly, provably maximal expressivity.

Expressivity theorem (Xu et al. 2019)

Message-passing GNNs distinguish at most what the **Weisfeiler–Leman** colour-refinement test distinguishes. WL-equivalent non-isomorphic graphs (e.g. certain strongly regular graphs) receive *identical* GNN outputs — provably indistinguishable, however trained.

- Remedies: higher-order (k -WL) GNNs, extra structural/positional features (e.g. Laplacian eigenvectors, cycle counts), subgraph methods — all trading cost for power.
- Moral for mathematicians: know the resolution of your instrument; if your invariant is beyond WL, vanilla GNNs cannot learn it exactly — an honest theorem, and a good source of research questions.



Every vertex has degree 2 with identical neighbourhoods at every round: WL, and so every message-passing GNN, returns the same answer for both. Yet $|\pi_0|$ is 1 and 2.

Point Clouds and $E(3)$ -Equivariance

Point sets in a physical space

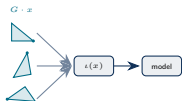
Data = $\{x_1, \dots, x_N\} \subset \mathbb{R}^3$ (+ features): surface samples, atomic configurations, lattice points.
Symmetries: S_N (ordering) and $E(3)$ (rotations, translations, reflections of space).

- **DeepSets:** $f(\{x_i\}) = \rho\left(\sum_i \eta(x_i)\right)$ characterises permutation-invariant functions — the minimal set architecture.
- **EGNN-style layers:** update invariant features via distances $\|x_i - x_j\|$, update positions along difference vectors $(x_i - x_j)$; $E(3)$ -equivariance from using only covariant ingredients (no coordinates ever appear bare).
- **Tensor-field/steerable networks:** features in $SO(3)$ -irreps (scalars, vectors, higher tensors) composed with Clebsch–Gordan products — full machinery for physics-grade equivariance.
- **Flagship applications:** molecular property prediction and interatomic potentials (state of the art is equivariant), protein structure, mesh processing; for us — learned energies/functionals on embedded point configurations (Tutorial 10).

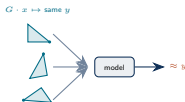
Three Roads to Symmetry — a Decision Guide

Incorporating a data symmetry G into learning

- 1 **Invariant features by hand:** feed the model G -invariants you compute (distances, spectra, characteristic numbers). Cheapest; interpretable; but limited to invariants you thought of, the model cannot discover new ones.
- 2 **Data augmentation:** train a generic model on G -translates. Easy; *approximate* invariance only; costly for large groups.
- 3 **Equivariant architecture:** exact symmetry, learned features, best data efficiency; highest implementation effort.



1. Orbit collapsed *before* learning — exact, but only the invariants you chose.



2. Orbit shown *during* training — easy, but invariance only approximate.



3. Symmetry inside *every* layer — exact by construction, most effort.

Practical implementation

In practice, combine: invariant features as inputs *to* an equivariant architecture, augmentation as a safety net. Then always *test* invariance numerically (apply g , compare outputs; it's one assert).

Applications in Geometry

- **Triangulations & move graphs:** 3-manifolds as plumbing graphs — a GNN decides homeomorphism, then an RL agent (Lecture 9) searches for the connecting sequence of Neumann moves; the output is a *checkable certificate*, not a prediction. Pachner graphs of triangulations await the same treatment. [2305.05966, 2405.09610]
- **Representation theory:** GNNs on Bruhat interval graphs predict Kazhdan–Lusztig coefficients; saliency on the trained model isolated the *hypercube decomposition* — conjecture, then theorem. [2111.15161]
- **Cluster algebras:** mutation-equivalence of quivers as graph classification; explainability independently recovered the type- $\tilde{D}$ criteria, and the hidden representations encoded type D unprompted. [2203.13847, 2411.07467]
- **Knots:** a *functor* from knots to graphs, then a GNN for invariants — functoriality is exactly the statement that the encoding respects the equivalence. [2305.16808]
- **Geometry $\rightarrow$ ML:** discrete Ricci curvature of edges identifies the bottlenecks causing over-squashing; curvature-guided rewiring repairs them (Lecture 11). [2111.14522]

Research interface

- **ML $\rightarrow$ Geometry:** Train well, then interrogate the representation
 $\Rightarrow$ a conjecture or a certificate you can check (never the network).
- **Geometry $\rightarrow$ ML:** Encode symmetry into design of the architecture
 $\Rightarrow$ failure informs on symmetry presence.

Summary

Key points

- Invariance and equivariance formalise “respecting symmetry”;
...equivariant hypothesis spaces gain data efficiency and exact correctness.
- Symmetric Architectures: layers = G -intertwiners + compatible nonlinearities;
...representation theory (Schur, irreps) computes the possibilities.
- CNNs and transformers were special cases;
...group/steerable CNNs extend to rotations and Lie groups.
- GNNs = permutation-equivariant message passing directly on graphs;
...expressivity is bounded by Weisfeiler–Leman.
- Three roads: invariant features, augmentation, equivariant architecture
...combine pragmatically, and always test the symmetry.

Next

Tutorial 10: Invariant vs non-invariant models on symmetric data; small GNNs.

Lecture 11: Manifolds, spectral methods, and diffusion models.